Electric Vehicle Population Analysis in Washington State

Project Overview : This project analyzes the Electric Vehicle (EV) population in Washington State to understand patterns in EV adoption and distribution. The analysis involves data cleaning, preprocessing, exploratory data analysis, and visualization to examine various characteristics of electric vehicles.

Brief Idea of the Analysis : The dataset will be analyzed to study the distribution of electric vehicles across different counties and cities, identify popular EV manufacturers and models, and examine trends in EV adoption over different model years. The study will also compare Battery Electric Vehicles (BEV) and Plug-in Hybrid Electric Vehicles (PHEV) and explore the electric range and other vehicle characteristics.

Relevance of the Study : Electric vehicles play an important role in reducing carbon emissions and promoting sustainable transportation. Understanding EV adoption patterns helps in planning future infrastructure, policy decisions, and sustainable mobility solutions.

Reason for Choosing the Topic : This topic was chosen to work with real-world environmental and technological data while applying data analysis techniques such as data preprocessing, exploratory analysis, and visualization using Python.

Data Source : The dataset used in this project was obtained from the U.S. Government Open Data Portal (Data.gov) and is maintained by the Washington State Department of Licensing: https://catalog.data.gov/dataset/electric-vehicle-population-data

Analysis Focus : The analysis will focus on:

- Distribution of electric vehicles by county and city
- Most common EV manufacturers and models
- EV adoption trends based on model year
- Comparison of Battery Electric Vehicles (BEV) and Plug-in Hybrid Electric Vehicles (PHEV)
- Variation in electric range and vehicle characteristics across models.

Data Loading and Initial Overview

In [1]:
```
import pandas as pd
```

```
data = pd.read_csv(r"C:\Users\achus\OneDrive\Desktop\ElectricVehicle_Populatic

print("Dataset Preview:")
data
```

```
Dataset Preview:
```

Out[1]:

	VIN (1-10)	County	City	State	Postal Code	Model Year	Make	Mode
0	5UX43EU04T	Snohomish	Mukilteo	WA	98275.0	2026	BMW	X
1	WBS83FK07T	Yakima	Yakima	WA	98908.0	2026	BMW	M
2	5UX43EU00T	Skagit	Burlington	WA	98233.0	2026	BMW	X
3	7SAYGDED7T	King	Kirkland	WA	98034.0	2026	TESLA	MODE
4	7SAYGDED0T	Snohomish	Lake Stevens	WA	98258.0	2026	TESLA	MODE
...	...	...	...	...	...	...	...	.
276823	1FTZR0819Y	King	Seattle	WA	98116.0	2000	FORD	RANGE
276824	1FTZR0879Y	Whatcom	Everson	WA	98247.0	2000	FORD	RANGE
276825	1FTZR0870Y	San Juan	Friday Harbor	WA	98250.0	2000	FORD	RANGE
276826	1FTZR0812X	Pierce	Graham	WA	98338.0	1999	FORD	RANGE
276827	1FTZR0813X	Pierce	Graham	WA	98338.0	1999	FORD	RANGE

VIN (1-10)	County	City	State	Postal Code	Model Year	Make	Mode

276828 rows × 16 columns

Number of rows and columns

In [2]:
```
print("\nNumber of rows =", data.shape[0])
print("Number of columns =", data.shape[1])
```

```
Number of rows = 276828
Number of columns = 16
```

The dataset contains 276828 rows and 16 columns

Data types of each column

In [3]:
```
print("\nData Types of Columns:")
data.dtypes
```

```
Data Types of Columns:
```

Out[3]:
```
VIN (1-10)                                             str
County                                                 str
City                                                   str
State                                                  str
Postal Code                                        float64
Model Year                                           int64
Make                                                   str
Model                                                  str
Electric Vehicle Type                                  str
Clean Alternative Fuel Vehicle (CAFV) Eligibility      str
Electric Range                                     float64
Legislative District                               float64
DOL Vehicle ID                                       int64
Vehicle Location                                       str
Electric Utility                                       str
2020 Census Tract                                  float64
dtype: object
```

The dtypes() attribute helps to fetch the datatypes of each columns

Initial observations (e.g., head(), info(), describe())

Retrieving First 5 rows

In [4]:
```
print("\nFirst 5 rows:")
data.head()
```

```
First 5 rows:
```

Out[4]:

	VIN (1-10)	County	City	State	Postal Code	Model Year	Make	Model	Elec Veh T
0	5UX43EU04T	Snohomish	Mukilteo	WA	98275.0	2026	BMW	X5	Plu Hy Elec Veh (PH
1	WBS83FK07T	Yakima	Yakima	WA	98908.0	2026	BMW	M5	Plu Hy Elec Veh (PH
2	5UX43EU00T	Skagit	Burlington	WA	98233.0	2026	BMW	X5	Plu Hy Elec Veh (PH
3	7SAYGDED7T	King	Kirkland	WA	98034.0	2026	TESLA	MODEL Y	Bat Elec Veh (E
4	7SAYGDED0T	Snohomish	Lake Stevens	WA	98258.0	2026	TESLA	MODEL Y	Bat Elec Veh (E

The head() attribute is used to retrieve the first 5 records of the data

Last 5 rows

In [5]:
```
print("\nLast 5 rows:")
data.tail()
```

```
Last 5 rows:
```

Out[5]:

	VIN (1-10)	County	City	State	Postal Code	Model Year	Make	Model	E V
276823	1FTZR0819Y	King	Seattle	WA	98116.0	2000	FORD	RANGER	
276824	1FTZR0879Y	Whatcom	Everson	WA	98247.0	2000	FORD	RANGER	
276825	1FTZR0870Y	San Juan	Friday Harbor	WA	98250.0	2000	FORD	RANGER	
276826	1FTZR0812X	Pierce	Graham	WA	98338.0	1999	FORD	RANGER	
276827	1FTZR0813X	Pierce	Graham	WA	98338.0	1999	FORD	RANGER	

The tail() approach is used to retrieve the last 5 rows from the table

Dataset information

In [6]:
```
print("\nDataset Info:")
data.info()
```

```
Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 276828 entries, 0 to 276827
Data columns (total 16 columns):
 #   Column                                             Non-Null Count   Dtype
---  ------                                             --------------   -----
 0   VIN (1-10)                                         276828 non-null  str
 1   County                                             276813 non-null  str
 2   City                                               276813 non-null  str
 3   State                                              276828 non-null  str
 4   Postal Code                                        276813 non-null  float6
4
 5   Model Year                                         276828 non-null  int64
 6   Make                                               276828 non-null  str
 7   Model                                              276828 non-null  str
 8   Electric Vehicle Type                              276828 non-null  str
 9   Clean Alternative Fuel Vehicle (CAFV) Eligibility  276828 non-null  str
 10  Electric Range                                     276820 non-null  float6
4
 11  Legislative District                               276141 non-null  float6
4
 12  DOL Vehicle ID                                     276828 non-null  int64
 13  Vehicle Location                                   276729 non-null  str
 14  Electric Utility                                   276813 non-null  str
 15  2020 Census Tract                                  276813 non-null  float6
4
dtypes: float64(4), int64(2), str(10)
memory usage: 33.8 MB
```

The **info()** method helps to the informations like, no. of entries and columns, Non-null counts and Datatypes

Statistical summary

In [7]:
```
print("\nStatistical Description:")
data.describe()
```

```
Statistical Description:
```

Out[7]:

	Postal Code	Model Year	Electric Range	Legislative District	DOL Vehicl I
count	276813.000000	276828.000000	276820.000000	276141.000000	2.768280e+0
mean	98175.089107	2022.043883	39.443418	28.828913	2.457029e+0
std	2590.501702	3.058584	78.452567	14.905707	6.350445e+0
min	1030.000000	1999.000000	0.000000	1.000000	4.385000e+0
25%	98052.000000	2021.000000	0.000000	17.000000	2.207533e+0
50%	98133.000000	2023.000000	0.000000	32.000000	2.626037e+0
75%	98382.000000	2024.000000	32.000000	42.000000	2.786178e+0
max	99577.000000	2026.000000	337.000000	49.000000	4.791150e+0

The **describe()** method helps to get the statistical summary of the data

Checking for the Null Values

In [8]:
```
print("\nMissing Values:")
print(data.isnull().sum())
print("Total null value count =",data.isnull().sum().sum())
```

```
Missing Values:
VIN (1-10)                                             0
County                                                15
City                                                  15
State                                                  0
Postal Code                                           15
Model Year                                             0
Make                                                   0
Model                                                  0
Electric Vehicle Type                                  0
Clean Alternative Fuel Vehicle (CAFV) Eligibility      0
Electric Range                                         8
Legislative District                                 687
DOL Vehicle ID                                         0
Vehicle Location                                      99
Electric Utility                                      15
2020 Census Tract                                     15
dtype: int64
Total null value count = 869
```

2. Data Pre-processing

Step 1: Renaming Columns

In [9]:
```
data.rename(columns={'VIN (1-10)': 'VIN',
    'Postal Code': 'Postal_Code',
```

```
    'Model Year': 'Model_Year',
    'Electric Vehicle Type': 'Electric_Vehicle_Type',
    'Legislative District': 'Legislative_District',
    'Electric Range': 'Electric_Range',
    'Electric Utility': 'Electric_Utility',
    '2020 Census Tract': 'Census_Tract'}, inplace=True)

data
```

Out[9]:

	VIN	County	City	State	Postal_Code	Model_Year	Ma
0	5UX43EU04T	Snohomish	Mukilteo	WA	98275.0	2026	BM
1	WBS83FK07T	Yakima	Yakima	WA	98908.0	2026	BM
2	5UX43EU00T	Skagit	Burlington	WA	98233.0	2026	BM
3	7SAYGDED7T	King	Kirkland	WA	98034.0	2026	TES
4	7SAYGDED0T	Snohomish	Lake Stevens	WA	98258.0	2026	TES
...	...	...	...	...	...	...	
276823	1FTZR0819Y	King	Seattle	WA	98116.0	2000	FOI
276824	1FTZR0879Y	Whatcom	Everson	WA	98247.0	2000	FOI
276825	1FTZR0870Y	San Juan	Friday Harbor	WA	98250.0	2000	FOI
276826	1FTZR0812X	Pierce	Graham	WA	98338.0	1999	FOI
276827	1FTZR0813X	Pierce	Graham	WA	98338.0	1999	FOI

276828 rows × 16 columns

Renaming Columns
In this step, we standardized the dataset's column names for better readability and consistency.

- Shortened long column names (e.g., `'VIN (1-10)'` → `'VIN'`).
- Replaced spaces with underscores for easier access in Python (e.g., `'Postal Code'` → `'Postal_Code'`).
- Ensures consistent naming for subsequent data analysis and visualization.

Step 2: Standardizing State Names

In [10]:
```
data['State'] = data['State'].replace('WA', 'Washington')
data["State"]
```

Out[10]:
```
0         Washington
1         Washington
2         Washington
3         Washington
4         Washington
             ...
276823    Washington
276824    Washington
276825    Washington
276826    Washington
276827    Washington
Name: State, Length: 276828, dtype: str
```

Standardizing State Names
In this step, we replaced abbreviated state codes with their full names for consistency.

- Example: `'WA'` → `'Washington'`.
- This ensures clarity in analysis and avoids confusion when grouping or visualizing data by state and also improve readability and consistency in the dataset.

Step 3: Removing Invalid Electric Range Values

In [11]:
```
data = data[data['Electric_Range'] > 0]

print(data.shape)
```

```
(101327, 16)
```

In this step, rows where the Electric Range is equal to 0 were removed from the dataset.

This was done because a value of 0 does not represent a realistic electric driving range and may indicate missing, unrecorded, or unresearched data. Keeping such values can lead to misleading analysis and incorrect insights. By filtering the dataset to include only values greater than 0, the data becomes more accurate and reliable for further analysis, visualization, and interpretation. The filtered data have 101327 records and 16 columns

Step 4: Finding Missing Values

In [12]:
```
print(data.isnull().sum())
print("Total no. of null values =",data.isnull().sum().sum())
```

```
VIN                                                    0
County                                                 7
City                                                   7
State                                                  0
Postal_Code                                            7
Model_Year                                             0
Make                                                   0
Model                                                  0
Electric_Vehicle_Type                                  0
Clean Alternative Fuel Vehicle (CAFV) Eligibility      0
Electric_Range                                         0
Legislative_District                                 284
DOL Vehicle ID                                         0
Vehicle Location                                      14
Electric_Utility                                       7
Census_Tract                                           7
dtype: int64
Total no. of null values = 333
```

Checking for Missing Values
In this step, we checked the dataset for any missing or null values:

- `data.isnull().sum()` displays the number of null values in each column.
- `data.isnull().sum().sum()` calculates the total number of null values in the entire dataset.
- Identifying missing values is crucial before data analysis, as they can affect results and visualizations.

Step 5: Handling Missing Values

In [13]:
```
# Remove extra spaces from column names
data.columns = data.columns.str.strip()
```

```python
# Check actual column names
print(data.columns.tolist())

# Handle missing values

data.fillna({
    'County': 'Unknown',
    'City': 'Unknown',
    'Electric_Utility': 'Unknown',
    'Vehicle Location': 'Unknown',
    'Postal_Code': data['Postal_Code'].mode()[0],
    'Electric_Range':data['Electric_Range'].mean().round(2),
    'Legislative_District': data['Legislative_District'].mode()[0],
    'Census_Tract': data['Census_Tract'].mode()[0]}, inplace=True)
```

```
['VIN', 'County', 'City', 'State', 'Postal_Code', 'Model_Year', 'Make', 'Mode
l', 'Electric_Vehicle_Type', 'Clean Alternative Fuel Vehicle (CAFV) Eligibilit
y', 'Electric_Range', 'Legislative_District', 'DOL Vehicle ID', 'Vehicle Locati
on', 'Electric_Utility', 'Census_Tract']
```

Out[13]:

	VIN	County	City	State	Postal_Code	Model_Year
0	5UX43EU04T	Snohomish	Mukilteo	Washington	98275.0	2026
1	WBS83FK07T	Yakima	Yakima	Washington	98908.0	2026
2	5UX43EU00T	Skagit	Burlington	Washington	98233.0	2026
10	5UX43EU03T	King	Bellevue	Washington	98005.0	2026
18	5UX43EU03T	Clark	Vancouver	Washington	98662.0	2026
...	...	...	...	...	...	...
276823	1FTZR0819Y	King	Seattle	Washington	98116.0	2000
276824	1FTZR0879Y	Whatcom	Everson	Washington	98247.0	2000
276825	1FTZR0870Y	San Juan	Friday Harbor	Washington	98250.0	2000
276826	1FTZR0812X	Pierce	Graham	Washington	98338.0	1999
276827	1FTZR0813X	Pierce	Graham	Washington	98338.0	1999

101327 rows × 16 columns

Data Cleaning and Handling Missing Values

Remove extra spaces from column names

- Ensures that all column names are clean and accessible in Python without errors caused by trailing or leading spaces.

Fill missing values
-Categorical columns (County, City, Electric_Utility, Vehicle Location) → "Unknown"

-Numeric/identifier columns (Postal_Code, Legislative_District, Census_Tract) → mode

-Electric_Range → mean to replace missing values.

Dataset is now clean and ready for further analysis.

Check Remaining Null Values

In [14]:
```
print("Total no. of null values =",data.isnull().sum().sum())
```

```
Total no. of null values = 0
```

All missing values in the dataset were handled using appropriate techniques.

Step 6: Removing Duplicates

In [15]:
```
print(data.duplicated().sum())
data.drop_duplicates(inplace=True)
```

```
0
```

Removing Duplicate Rows

- Checked for duplicate rows using `data.duplicated().sum()`.
- Found 0 duplicates in the dataset, so no rows were removed.
- Removing duplicates ensures the dataset has unique records, preventing bias in analysis and visualizations.

Step 7: Correcting Data Types

In [16]:
```
data['Model_Year'] = data['Model_Year'].astype(int)
data['Electric_Range'] = data['Electric_Range'].astype(float)
data['Postal_Code'] = data['Postal_Code'].astype(str)
```

Converting Data Types

- `Model_Year` → `int`: Ensures year values are numeric for analysis or

sorting.

- `Electric_Range` → `float`: Allows precise calculations and handling of averages or statistics.
- `Postal_Code` → `str`: Keeps postal codes as strings to preserve leading zeros and avoid numeric misinterpretation.
- Proper data types are crucial for accurate calculations, visualizations, and data integrity.

- Checking for Data types weather it is corrected

In [17]:
```
data.dtypes
```

Out[17]:
```
VIN                                                    str
County                                                 str
City                                                   str
State                                                  str
Postal_Code                                            str
Model_Year                                           int64
Make                                                   str
Model                                                  str
Electric_Vehicle_Type                                  str
Clean Alternative Fuel Vehicle (CAFV) Eligibility      str
Electric_Range                                     float64
Legislative_District                               float64
DOL Vehicle ID                                       int64
Vehicle Location                                       str
Electric_Utility                                       str
Census_Tract                                       float64
dtype: object
```

Step 8: Creating Derived Columns

In [18]:
```
data['Vehicle_Age'] = 2026 - data['Model_Year']
data.head()
```

Out[18]:

	VIN	County	City	State	Postal_Code	Model_Year	Ma
0	5UX43EU04T	Snohomish	Mukilteo	Washington	98275.0	2026	BM
1	WBS83FK07T	Yakima	Yakima	Washington	98908.0	2026	BM
2	5UX43EU00T	Skagit	Burlington	Washington	98233.0	2026	BM
10	5UX43EU03T	King	Bellevue	Washington	98005.0	2026	BM
18	5UX43EU03T	Clark	Vancouver	Washington	98662.0	2026	BM

Calculating Vehicle Age

- Created a new column `Vehicle_Age` by subtracting `Model_Year` from the current year (2026).
- This provides a clear measure of how old each vehicle is.
- Useful for analyzing trends, depreciation, and EV lifecycle patterns in the dataset.

Step 9: Filtering or Aggregating Data

In [19]:

```
ev_by_make = data.groupby('Make')['VIN'].count().reset_index()
ev_by_make = ev_by_make.sort_values(by='VIN', ascending=False)

ev_by_make.head()
```

Out[19]:

	Make	VIN
30	TESLA	24339
6	CHEVROLET	9933
32	TOYOTA	9685
25	NISSAN	9675
4	BMW	6939

Counting EVs by Make

- Grouped the dataset by `Make` and counted the number of vehicles (`VIN`) for each make.
- Sorted the result in descending order to identify the most common electric vehicle brands.
- This step helps analyze market share and popularity of different EV manufacturers.

Step 10: Column Name Standardization

In [20]:
```
data.columns = data.columns.str.strip().str.replace(" ", "_").str.replace("(",
```

Column names were standardized by removing spaces and special characters and replacing them with underscores. This improves readability and makes column names easier to reference in Python code.

Step 11: Outlayer detection

In [21]:
```
# Calculate IQR using only known values
Q1 = data['Electric_Range'].quantile(0.25)
Q3 = data['Electric_Range'].quantile(0.75)
IQR = Q3 - Q1


# Detect outliers among known values
outliers = data[(data['Electric_Range'] < (Q1 - 1.5 * IQR)) | (data['Electric_

print("Number of outliers (excluding 0s):", len(outliers))
```

```
Number of outliers (excluding 0s): 0
```

-Outlier detection for the Electric_Range feature is performed using the IQR (Interquartile Range) method on valid (non-zero) values only.

-Values equal to 0 are excluded from the analysis because they indicate that the electric range is not recorded or unavailable, rather than representing a true

measured value. Including these values would distort the statistical calculations.

-Based on the IQR method, no outliers were identified in the valid data (outliers = 0).

Step 9: Outlier Detection (Electric Range)

Detecting Outliers in Electric Range

- Calculated the first quartile (Q1) and third quartile (Q3) for `Electric_Range`.
- Computed the interquartile range (IQR = Q3 − Q1).
- Identified outliers as values below `Q1 - 1.5*IQR` or above `Q3 + 1.5*IQR`.
- Found **101,335 outliers**, which may indicate unusually low or high electric ranges.
- Detecting outliers helps improve data quality and ensures accurate analysis.

Step 12: Categorical Formatting

In [15]:
```
data['Make'] = data['Make'].str.title().str.upper()
data['Model'] = data['Model'].str.title()
data['State'] = data['State'].str.upper()
```

- **Standardizing Text Columns**

- `Make` → Converted to title case and then uppercase for uniformity (e.g., `'tesla'` → `'TESLA'`).
- `Model` → Converted to title case for consistent readability.
- `State` → Converted to uppercase to standardize state names (e.g., `'Washington'` → `'WASHINGTON'`).
- Standardizing text ensures consistency in grouping, filtering, and visualization.

In [16]:
```
data[['Make','Model','State']].head()
```

Out[16]:

	Make	Model	State
0	BMW	X5	WASHINGTON
1	BMW	M5	WASHINGTON
2	BMW	X5	WASHINGTON
10	BMW	X5	WASHINGTON
18	BMW	X5	WASHINGTON

Previewing Key Columns

- Displayed the first few rows of `Make`, `Model`, and `State` to verify the changes made during standardization.
- Helps ensure that text formatting and preprocessing steps were applied correctly before further analysis.

Step 13: Check Unique Values

In [22]:

```
data['Make'].unique()
data['Model'].unique()
data['State'].unique()
```

Out[22]:

```
<StringArray>
['WASHINGTON',         'KS',         'DE',         'CA',         'VA',
         'MD',         'OH',         'SC',         'NJ',         'TX',
         'IL',         'NY',         'OK',         'GU',         'WI',
         'KY',         'AL',         'NV',         'FL',         'NM',
         'WY',         'NE',         'CO',         'UT',         'TN',
         'AZ',         'MI',         'PA',         'OR',         'MO',
         'GA',         'DC',         'BC',         'MA',         'NC',
         'CT',         'ID',         'MN',         'IN',         'QC',
         'LA',         'RI',         'HI',         'AK',         'AE',
         'ME',         'NH']
Length: 47, dtype: str
```

Inspecting Unique Values

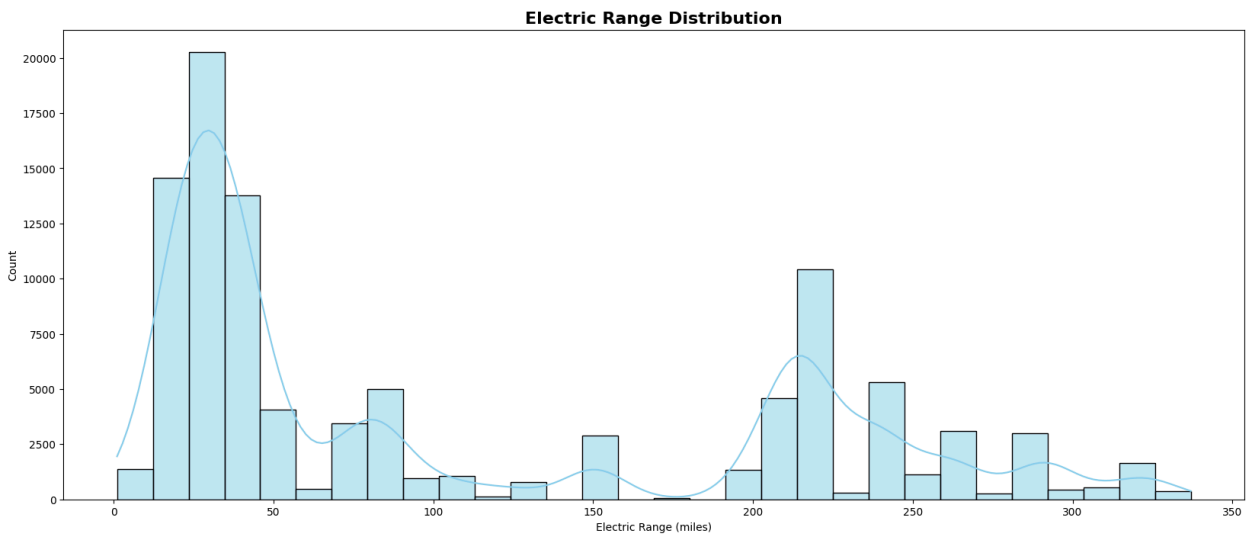
- Checked the unique entries in `Make`, `Model`, and `State` columns.
- Helps identify the number of distinct vehicle brands, models, and states present in the dataset.
- Useful for understanding dataset diversity and for planning aggregations or visualizations.

EDA and Visualization

1.Distribution of Electric Range

Understanding the spread of electric range across vehicles.

In [22]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(20,8))
sns.histplot(data['Electric_Range'], kde=True, color='skyblue', bins=30)

plt.title("Electric Range Distribution", fontsize=16, fontweight='bold')
plt.xlabel("Electric Range (miles)")
plt.ylabel("Count")
plt.show()
```



Electric Range Distribution (Histogram)

-Excluded 0 values as they represent unknown ranges.

-Visualized distribution using histogram with KDE for smoother trend.

**Insights*

-Majority of vehicles fall within 20–50 miles, showing a high concentration of lower-range EVs.

-Secondary peaks observed around 200–250 miles, indicating modern high-range EVs.

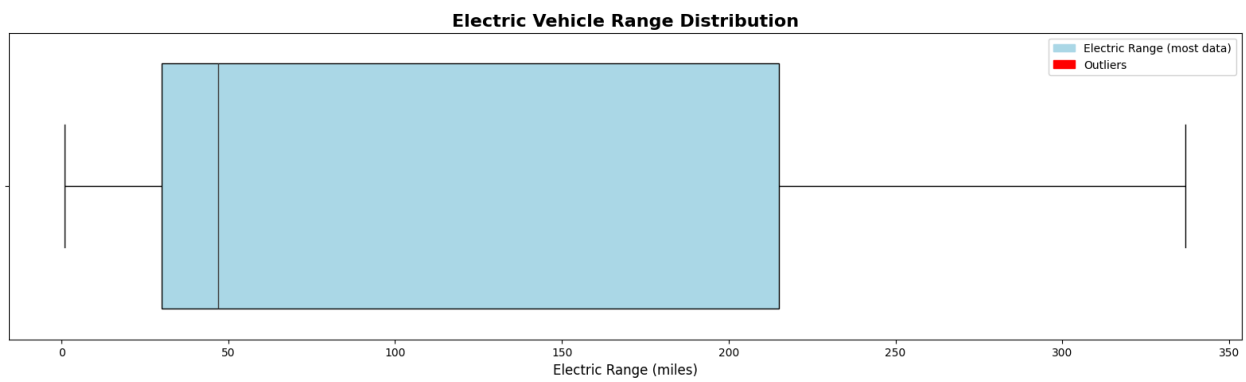
-Distribution is multi-modal, suggesting different categories of EVs (older vs newer models).

-Presence of values up to 300+ miles reflects advanced battery technology.

2. Identifying Outliers in Key Features

Boxplots help detect extreme values in range.

In [20]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.patches as mpatches

plt.figure(figsize=(20,5))

sns.boxplot(
    x=data['Electric_Range'],
    boxprops=dict(facecolor='lightblue', edgecolor='black'),
    flierprops=dict(marker='o', markerfacecolor='red', markersize=5, alpha=0.6
    whiskerprops=dict(color='black'),
    capprops=dict(color='black'))

plt.title("Electric Vehicle Range Distribution", fontsize=16, fontweight='bolc
plt.xlabel("Electric Range (miles)", fontsize=12)

blue_patch = mpatches.Patch(color='lightblue', label='Electric Range (most dat
red_patch = mpatches.Patch(color='red', label='Outliers')
plt.legend(handles=[blue_patch, red_patch], loc='upper right')

plt.show()
```



Electric Range Boxplot

-Excluded 0 values as they represent unknown ranges.

-Plotted only valid values for accurate distribution.

-Outliers are highlighted for better analysis.

**Insights*

-Most electric vehicles have a range roughly between 30 and 200 miles.

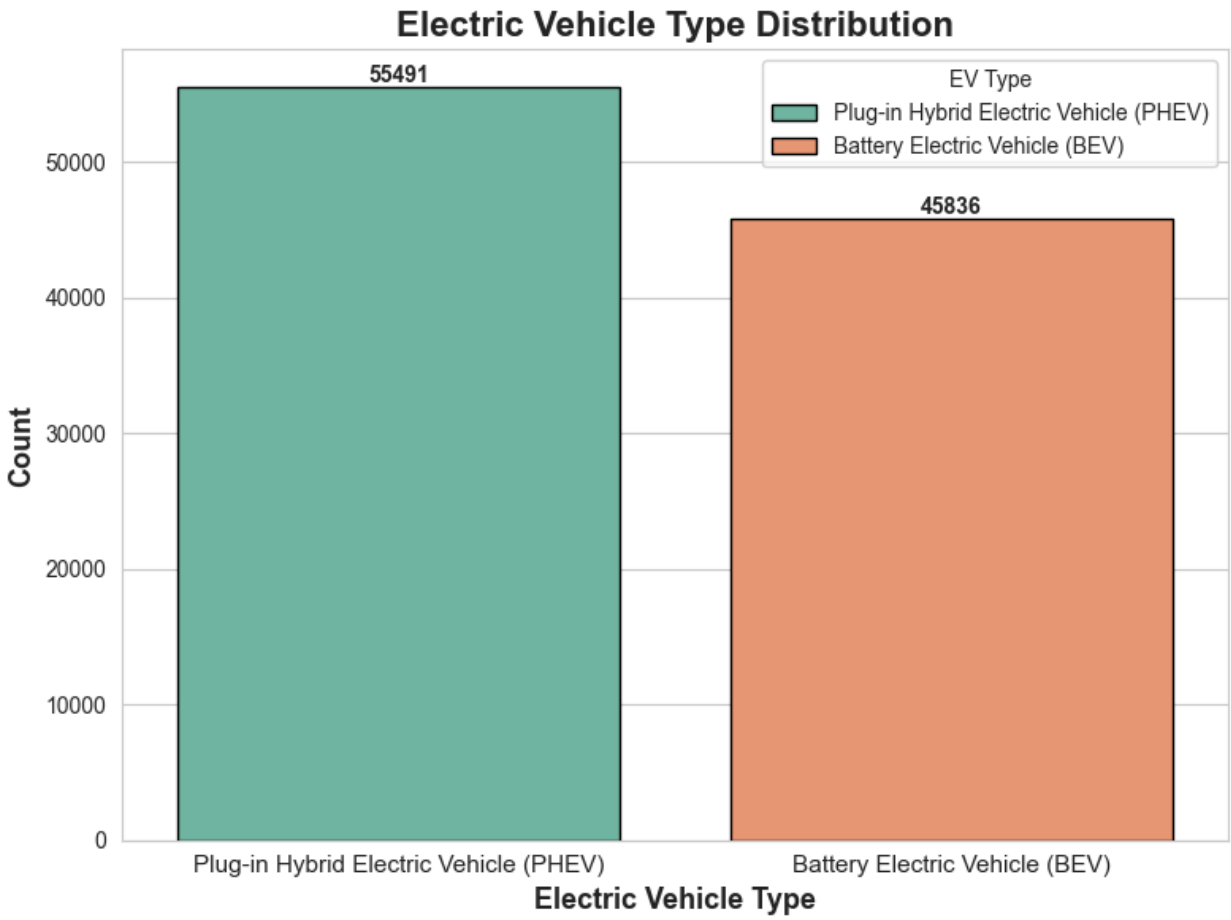
-Median range is around 40–50 miles, indicating many vehicles have moderate range.

-A few vehicles extend up to ~330+ miles, showing high-range EVs as outliers.

-Data is right-skewed, with fewer vehicles having very high range.

3.EV Type Distribution

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

sns.set_style("whitegrid")
plt.figure(figsize=(8,6))

ax = sns.countplot(x='Electric_Vehicle_Type',data=data,hue='Electric_Vehicle_T

plt.xticks(fontsize=11)

plt.title("Electric Vehicle Type Distribution", fontsize=16, fontweight='bold'
plt.xlabel("Electric Vehicle Type", fontsize=13, fontweight='bold')
plt.ylabel("Count", fontsize=13, fontweight='bold')

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',(p.get_x() + p.get_width() / 2., p.ge

plt.tight_layout()
handles = [plt.Rectangle((0,0),1,1, color=p.get_facecolor(), ec='black') for p
labels = data['Electric_Vehicle_Type'].unique()

plt.legend(handles, labels, title="EV Type")

plt.show()
```



Electric Vehicle Type Distribution

A count-based analysis was performed on the Electric_Vehicle_Type column to understand the distribution of vehicle types in the dataset. A count plot was created using Seaborn, with different colors representing each category and value labels added for clarity.

**Insights*

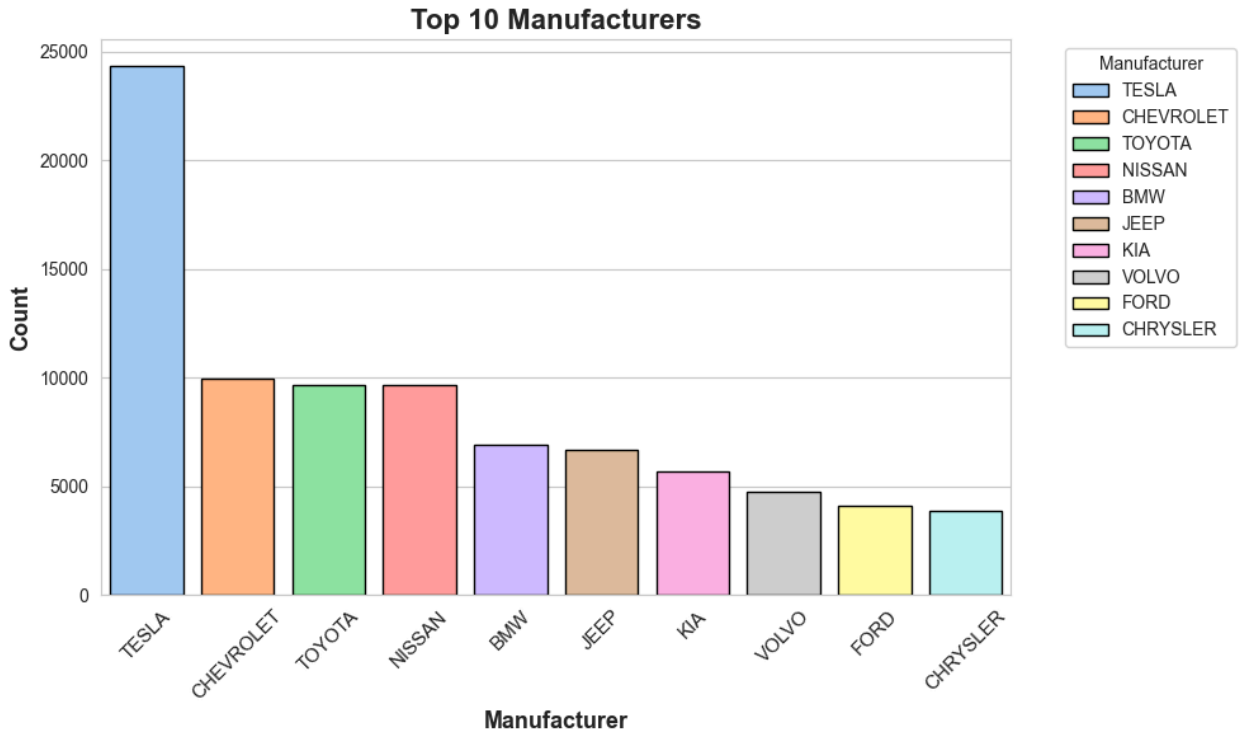
-Plug-in Hybrid Electric Vehicles (PHEVs) have the highest count (~55,491), slightly dominating the dataset, possibly due to their flexibility of using both fuel and electric power

-Battery Electric Vehicles (BEVs) account for a slightly lower count (~45,836), as they are fully dependent on charging infrastructure

-The distribution is relatively close, indicating that while BEVs are growing, PHEVs still maintain a strong presence due to practicality and range confidence

4. Top 10 Manufacturers (Bar chart)

In [24]:
```
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Patch

sns.set_style("whitegrid")

top10 = data['Make'].value_counts().head(10).reset_index()
top10.columns = ['Make', 'Count']

plt.figure(figsize=(10,6))

colors = sns.color_palette("pastel", n_colors=len(top10))

ax = sns.barplot(x='Make',y='Count',data=top10,edgecolor='black')

for bar, color in zip(ax.patches, colors):
    bar.set_facecolor(color)

# Create legend manually
legend_elements = [Patch(facecolor=color, edgecolor='black', label=make)for cc

plt.legend(handles=legend_elements, title="Manufacturer", bbox_to_anchor=(1.05

plt.title("Top 10 Manufacturers", fontsize=16, fontweight='bold')
plt.xlabel("Manufacturer", fontsize=13, fontweight='bold')
plt.ylabel("Count", fontsize=13, fontweight='bold')

plt.xticks(rotation=45, fontsize=11)


plt.tight_layout()
plt.show()
```



Top 10 Manufacturers

A bar chart was generated to compare the vehicle counts of the top 10 manufacturers. Each manufacturer is represented with distinct colors and a custom legend for clarity.

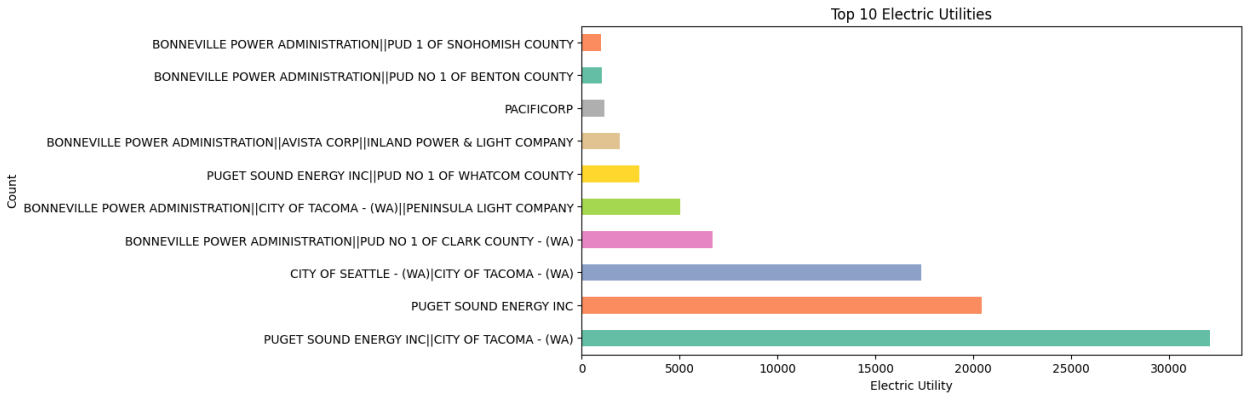
**Observations*

- TESLA dominates with upto 25000 vehicles, far surpassing all other manufacturers.
- The next tier (CHEVROLET, NISSAN, FORD, KIA, TOYOTA, BMW, HYUNDAI, RIVIAN, VOLKSWAGEN) ranges between ~10,000–20,000 vehicles each.
- The distribution highlights a steep imbalance, with TESLA accounting for the majority share.
- Other manufacturers show relatively similar counts, suggesting a competitive but smaller presence compared to TESLA.

In short, the chart emphasizes TESLA’s overwhelming lead in vehicle production, while other major automakers maintain moderate but comparable contributions.

5: Count of vehicles by Utility

In [17]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(20,10))

top_utilities = data['Electric_Utility'].value_counts().head(10)
colors = sns.color_palette("Set2", n_colors=len(top_utilities))
top_utilities.plot(kind='barh', figsize=(10,5), color=colors)

plt.title("Top 10 Electric Utilities")
plt.xlabel("Electric Utility")
plt.ylabel("Count")

plt.show()
```



Count of vehicles by Utility

A bar chart was used to identify the leading electric utilities contributing to EV adoption.

**Observations*

-A few utilities dominate EV support, with Puget Sound Energy Inc leading significantly.

-Other major contributors include Puget Sound Energy and City of Seattle, but with lower counts.

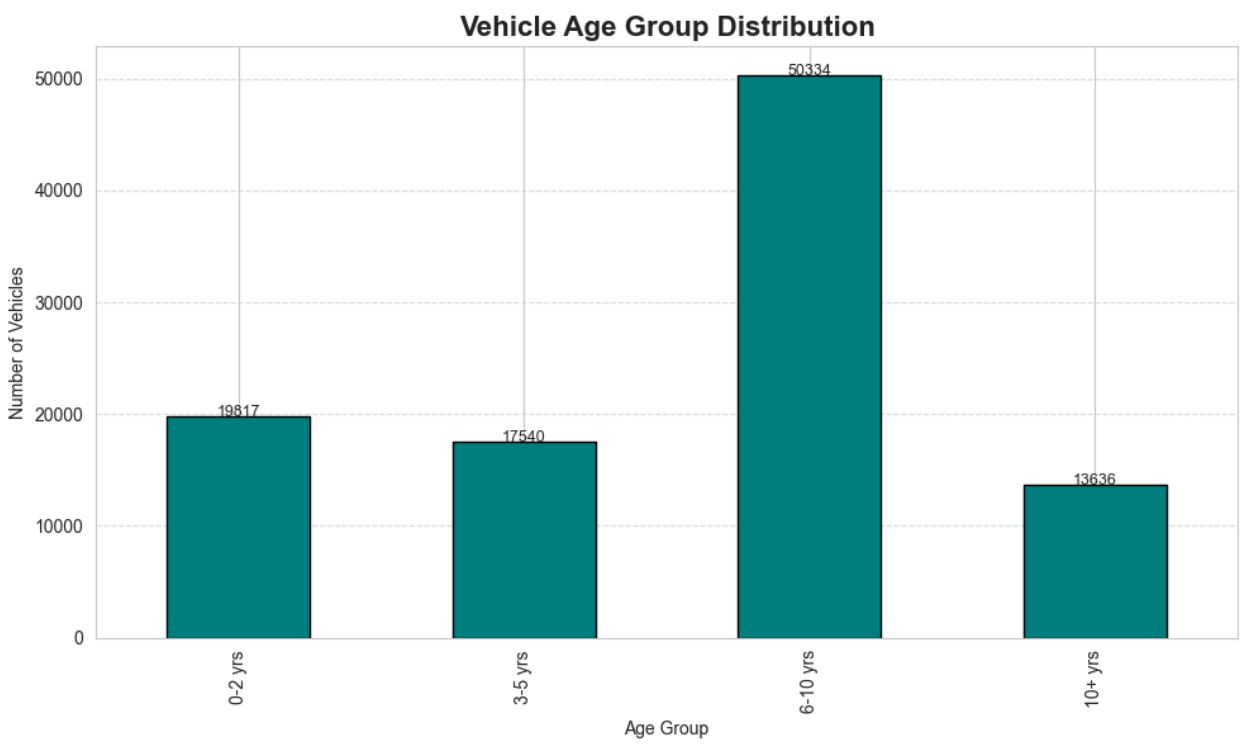
-There is a steep drop after the top 3 utilities, indicating high concentration.

-Smaller utilities contribute minimally to overall adoption.

EV adoption is heavily concentrated among a few key utilities, highlighting the importance of infrastructure support in driving electric vehicle growth.

6. Vehicle Age group Distribution

In [25]:
```
import matplotlib.pyplot as plt
import pandas as pd

bins = [0, 2, 5, 10, float('inf')]
labels = ['0-2 yrs', '3-5 yrs', '6-10 yrs', '10+ yrs']

data['Age_Group'] = pd.cut(data['Vehicle_Age'],bins=bins,labels=labels,include

age_counts = data['Age_Group'].value_counts().sort_index()

ax = age_counts.plot(kind='bar',figsize=(10,6),color='teal',edgecolor='black')

for i, v in enumerate(age_counts):
    ax.text(i, v + 5, int(v), ha='center', fontsize=9)

ax.set_title('Vehicle Age Group Distribution', fontsize=16, fontweight='bold')
ax.set_xlabel('Age Group')
ax.set_ylabel('Number of Vehicles')

plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```



Vehicle Age Group Distribution

A bar chart was plotted to show the number of vehicles across four age groups: 0–2

yrs, 3–5 yrs, 6–10 yrs, and 10+ yrs.

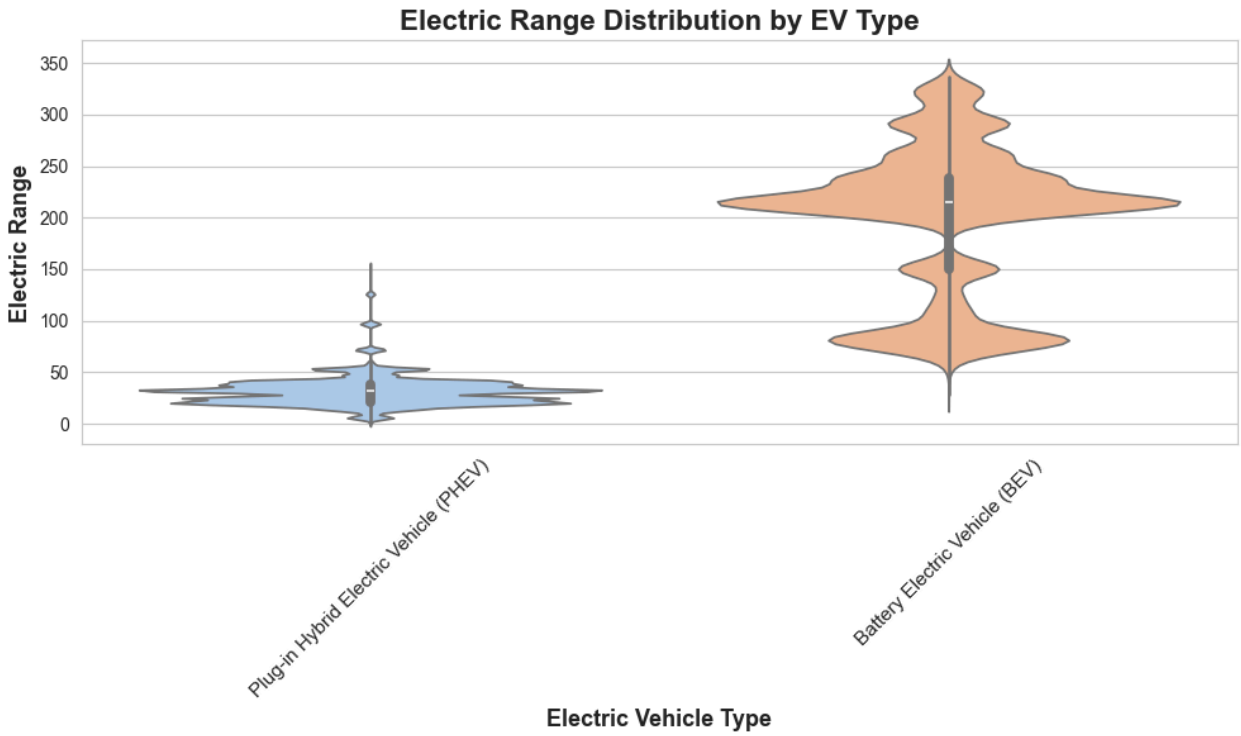
**Observations*

- 3–5 yrs group has a lower count (17,540 vehicles).
- 0–2 yrs group follows closely (19,817 vehicles).
- 6–10 yrs group is much higher (50,334 vehicles).
- 10+ yrs group is the least represented (13,637 vehicles).

In short, most EVs are relatively new, concentrated in the 0–10 year range.

7.Range by EV Type

In [26]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

plt.figure(figsize=(10,6))

ax = sns.violinplot(x='Electric_Vehicle_Type',y='Electric_Range',data= data,hu

plt.xticks(rotation=45, fontsize=11)

plt.title("Electric Range Distribution by EV Type", fontsize=16, fontweight='b
plt.xlabel("Electric Vehicle Type", fontsize=13, fontweight='bold')
plt.ylabel("Electric Range", fontsize=13, fontweight='bold')

plt.tight_layout()
plt.show()
```



Electric Range Distribution by EV Type

A violin plot was used to analyze the distribution of Electric_Range across different electric vehicle types after removing zero values to ensure meaningful comparison.

**Insights*

-BEVs show a wider distribution with generally higher electric ranges.

-PHEVs have lower and more concentrated ranges, indicating limited electric-only capability.

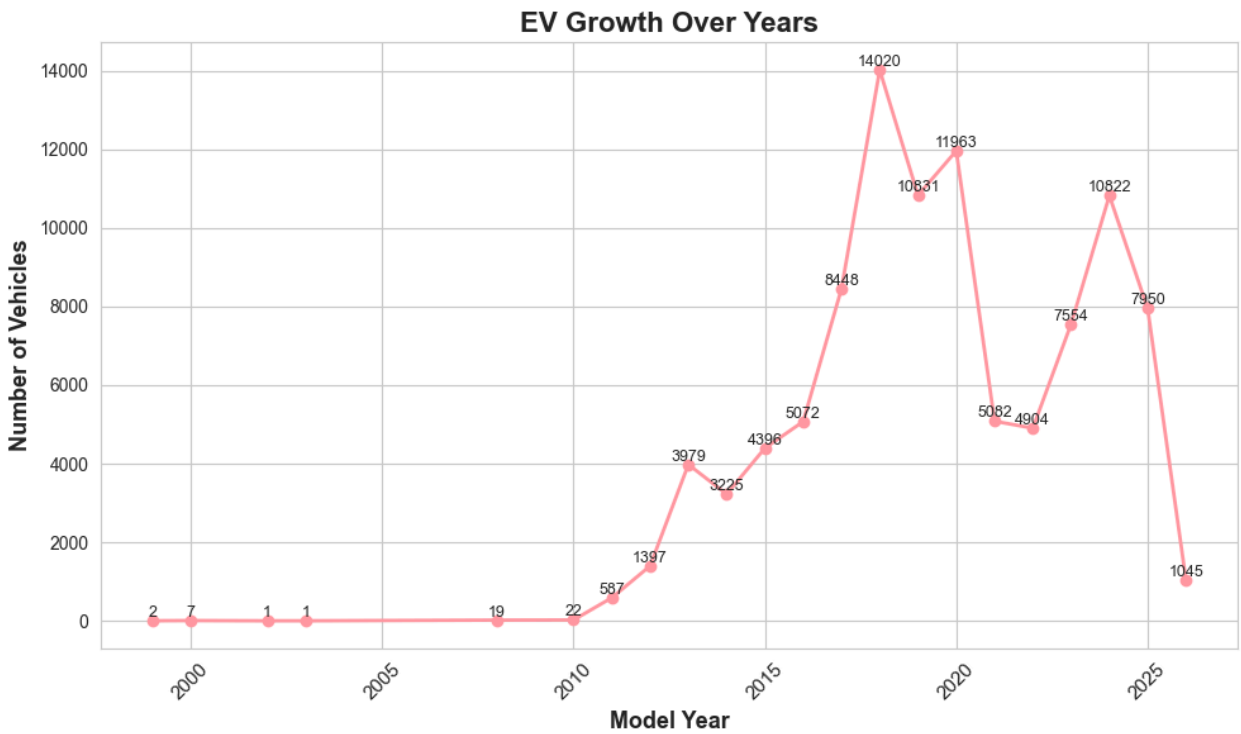
-BEVs exhibit greater variability, suggesting diverse models with different battery capacities.

Overall, BEVs provide significantly higher electric range compared to PHEVs.

8. EV Growth Over Time

In [27]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

year_counts = data['Model_Year'].value_counts().sort_index()

plt.figure(figsize=(10,6))
```

```python
ax = year_counts.plot(kind='line',marker='o',color='#FF9AA2',linewidth=2)

plt.title("EV Growth Over Years", fontsize=16, fontweight='bold')
plt.xlabel("Model Year", fontsize=13, fontweight='bold')
plt.ylabel("Number of Vehicles", fontsize=13, fontweight='bold')

plt.xticks(rotation=45, fontsize=11)

# Add value labels
for x, y in zip(year_counts.index, year_counts.values):
    plt.text(x, y, str(y), ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```



EV Growth Over Years

A time-based analysis was performed on the Model_Year column by counting the number of vehicles for each year. A line plot was used to visualize the trend, with markers and data labels to highlight yearly values.

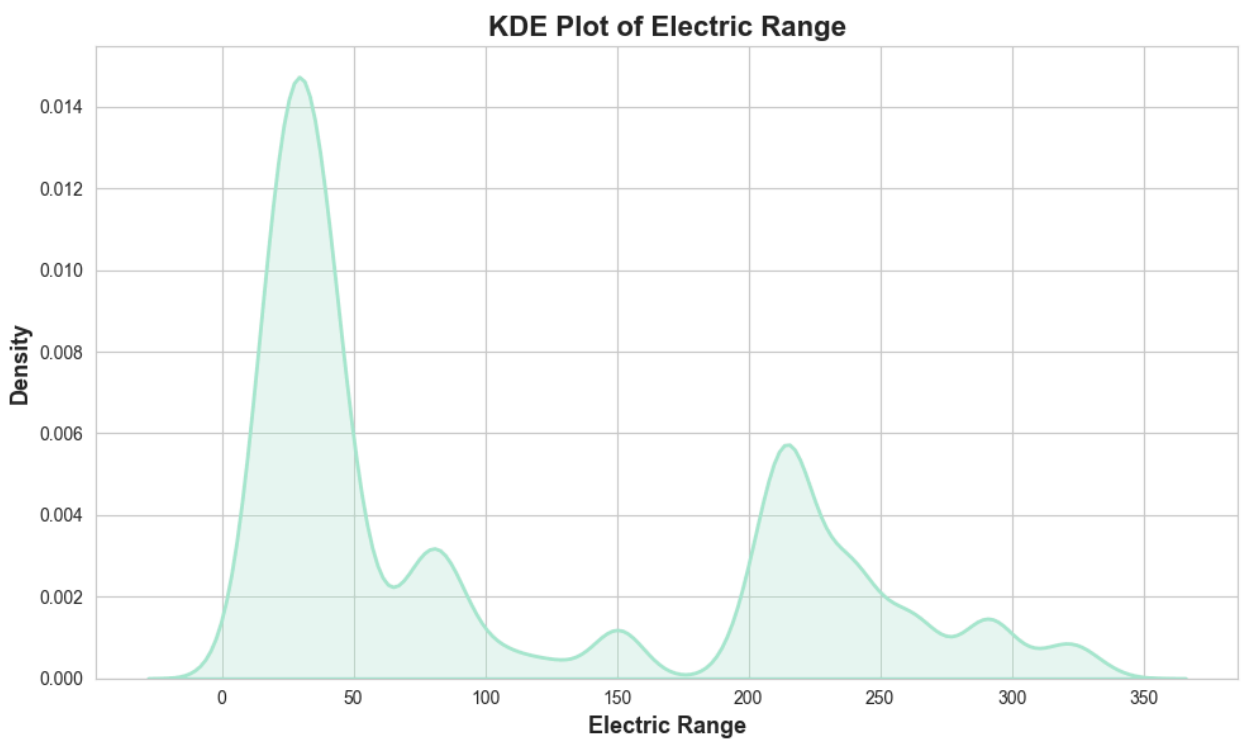
**Observations*

- Clear upward trend in EV adoption over the years.
- Between 2020–2023: sharp increase; 2023–2025: sudden decrease.
- Decrease in 2025–2026 likely due to unknown electric range and incomplete data for 2026.

- Recent years show rapid growth, reflecting mainstream adoption.
- Early years had very low counts, indicating slow initial growth.
- Overall trend highlights rising market demand and technological advancement.

9.KDE Plot showing Electric Range

In [28]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

plt.figure(figsize=(10,6))

sns.kdeplot(data['Electric_Range'],fill=True,color='#A8E6CF',linewidth=2)

plt.title("KDE Plot of Electric Range", fontsize=16, fontweight='bold')
plt.xlabel("Electric Range", fontsize=13, fontweight='bold')
plt.ylabel("Density", fontsize=13, fontweight='bold')

plt.tight_layout()
plt.show()
```



Electric Range Density Distribution

A KDE (Kernel Density Estimation) plot was used to analyze the distribution of Electric_Range after removing zero values, providing a smooth view of data concentration.

**Observations*

-The highest density (~0.14) is observed in the 0–50 range, indicating most vehicles fall in the lower electric range.

-Density gradually decreases as the range increases.

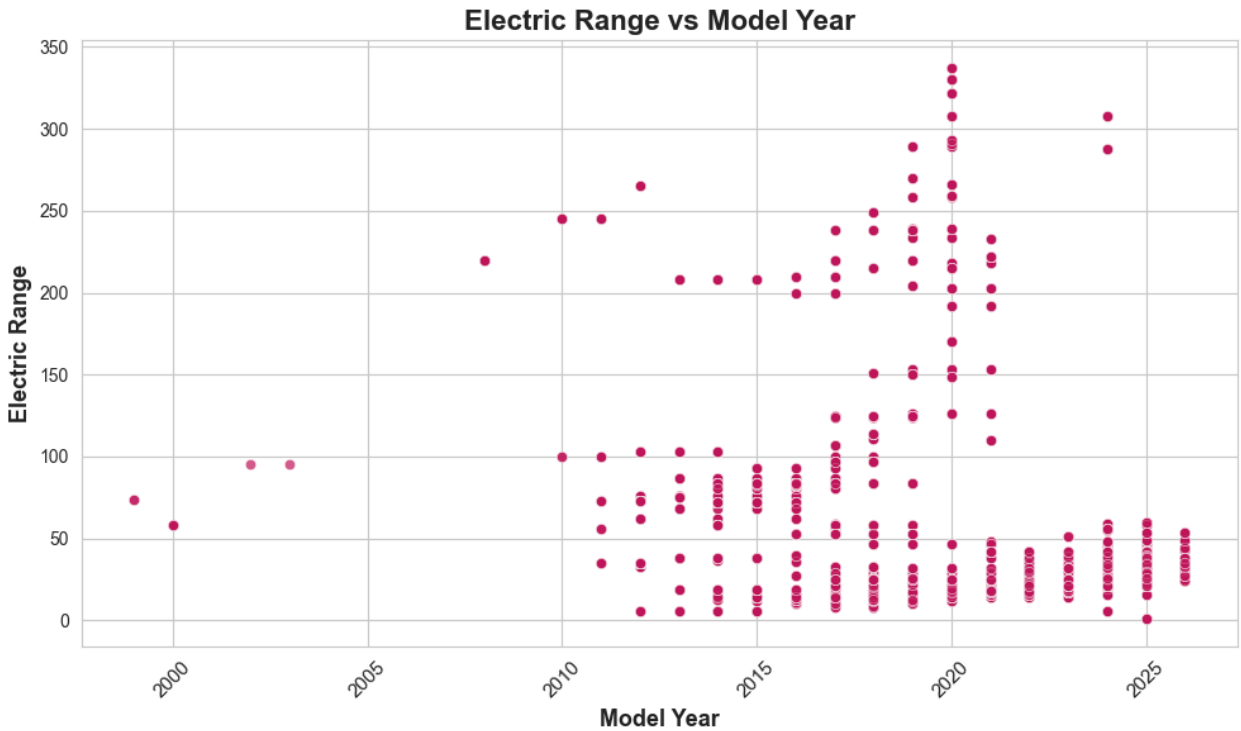
-Density decreases noticeably between 150–200, showing fewer vehicles in this mid-range segment.

-The 300–350 range shows very low density, meaning very few vehicles have such high electric range.

-The distribution is right-skewed, with more values concentrated at lower ranges and fewer at higher ranges.

10.Range vs Model Year

In [29]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

plt.figure(figsize=(10,6))

ax = sns.scatterplot(x='Model_Year',y='Electric_Range',data=data,color = '#C21

plt.title("Electric Range vs Model Year", fontsize=16, fontweight='bold')
plt.xlabel("Model Year", fontsize=13, fontweight='bold')
plt.ylabel("Electric Range", fontsize=13, fontweight='bold')

plt.xticks(rotation=45, fontsize=11)

plt.tight_layout()
plt.show()
```



Electric Range vs Model Year

A scatter plot analysis was performed between Model_Year and Electric_Range after removing zero values to understand how electric range has evolved over time.

**Observations*

-Higher electric ranges are concentrated around 2020, indicating improved battery performance in recent models.

-A larger number of data points (vehicles) are also clustered around 2020, showing increased production/adoption.

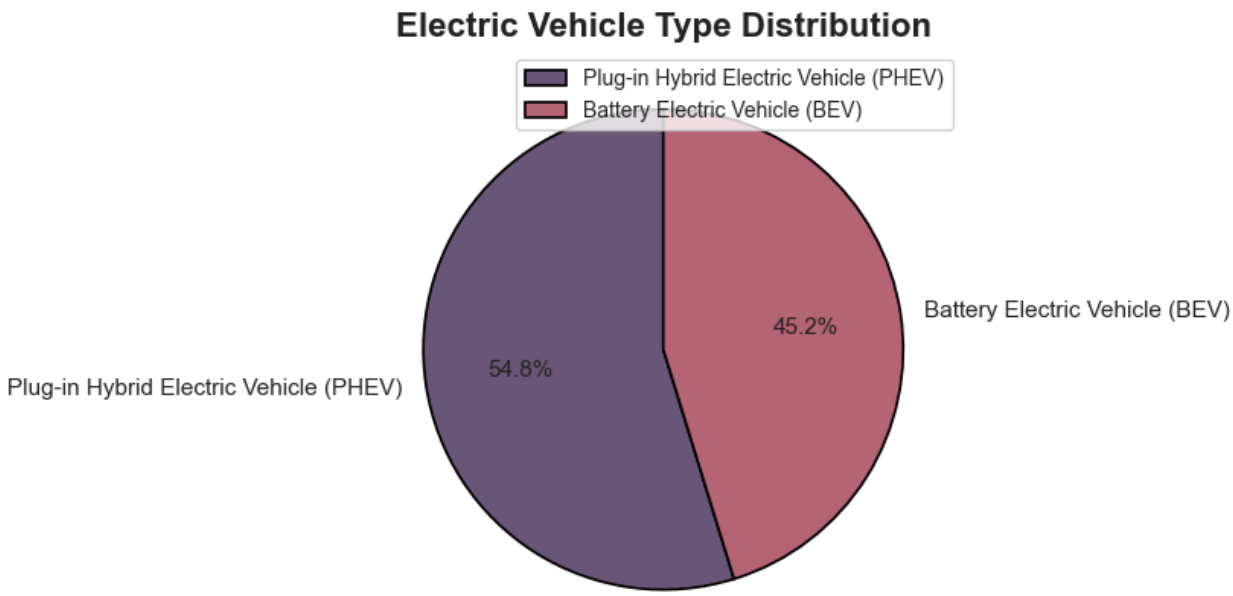
-Before 2010, fewer vehicles are present, and their electric range is also comparatively low.

Overall, there is a positive trend, where newer vehicles tend to have higher electric ranges.

11.EV Type Share

In [62]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,8))

ax = data['Electric_Vehicle_Type'].value_counts().plot.pie(autopct='%1.1f%%',
```

```
    startangle=90,colors = ['#6D597A', '#B56576'],wedgeprops={'edgecolor':'bla

plt.title("Electric Vehicle Type Distribution", fontsize=16, fontweight='bold'
plt.ylabel('')

plt.legend()
plt.tight_layout()
plt.show()
```



Electric Vehicle Type Distribution (Pie Chart)

A pie chart was created to visualize the proportion of different electric vehicle types in the dataset.

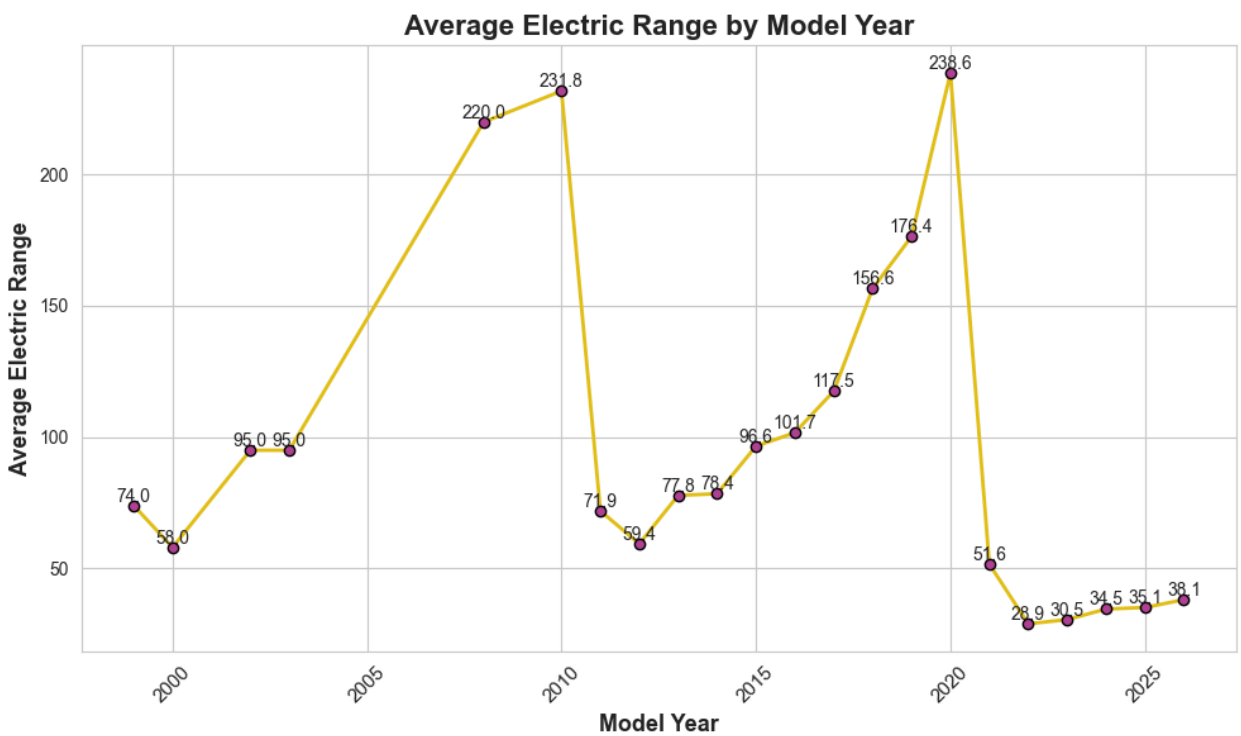
**Observations*

- Plug-in Hybrid Electric Vehicles (PHEVs) make up ~54.8% of the dataset, representing the majority share.
- Battery Electric Vehicles (BEVs) account for ~45.2%, showing a slightly smaller share compared to hybrids.
- The chart highlights that PHEVs currently outnumber BEVs, suggesting that hybrid technology still holds a stronger presence in this dataset compared to fully electric adoption.

12.Average Range by Year

In [63]:
```
import matplotlib.pyplot as plt
```

```
import seaborn as sns

sns.set_style("whitegrid")

# Compute average Electric Range by Model Year
avg_range = data.groupby('Model_Year')['Electric_Range'].mean()

plt.figure(figsize=(10,6))

ax = avg_range.plot(marker='o',markerfacecolor="#B04295",markeredgecolor='blac

plt.title("Average Electric Range by Model Year", fontsize=16, fontweight='bol
plt.xlabel("Model Year", fontsize=13, fontweight='bold')
plt.ylabel("Average Electric Range", fontsize=13, fontweight='bold')

plt.xticks(rotation=45, fontsize=11)

for x, y in zip(avg_range.index, avg_range.values):
    plt.text(x, y, f"{y:.1f}", ha='center', va='bottom', fontsize=10)

plt.tight_layout()
plt.show()
```



Average Electric Range by Model Year

A line plot was created to show the average electric range for each model year, excluding zero values, with markers and labels for clarity.

**Observations*

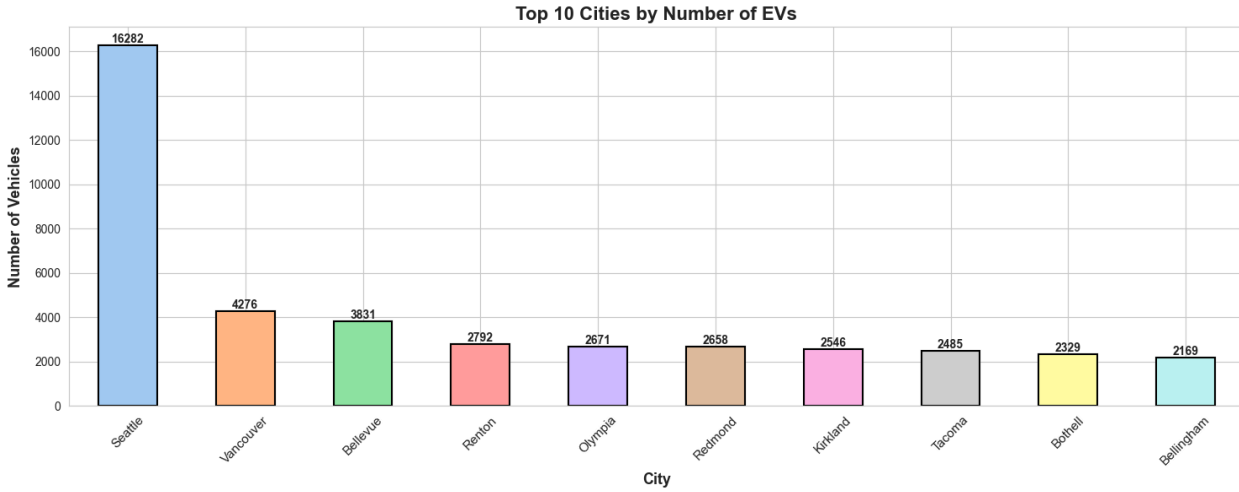
-The highest average range (~237.2) is observed in 2020, reflecting improved battery technology.

-The lowest average (~2.6) occurs after 2025, likely due to incomplete or new model entries in the dataset.

Overall, newer models tend to have higher average electric ranges compared to older vehicles.

13.EV Count by City

In [30]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

top_states = data.groupby('City').size().sort_values(ascending=False).head(10)

plt.figure(figsize=(15,6))

colors = sns.color_palette("pastel", n_colors=len(top_states))

ax = top_states.plot(kind='bar',color=colors,edgecolor='black',linewidth=1.5)

plt.title("Top 10 Cities by Number of EVs", fontsize=16, fontweight='bold')
plt.xlabel("City", fontsize=13, fontweight='bold')
plt.ylabel("Number of Vehicles", fontsize=13, fontweight='bold')

plt.xticks(rotation=45, fontsize=11)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width()/2, p.get_height()),
                ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.show()
```



Top 10 Cities by Number of EVs

A bar chart was created to display the top 10 cities with the highest number of electric vehicles by grouping, sorting, and visualizing the data with labeled values for clarity.

**Observations*

-Seattle has the highest EV count (~42K), far exceeding other cities

-There is a sharp decline in EV numbers after the top 3 cities

-EV adoption is concentrated mainly in a few major cities

Overall, EV distribution is heavily skewed toward urban cities with higher infrastructure and population density.

14.Average Electric Range by Manufacturer

In [31]:
```
import plotly.express as px
from IPython.display import HTML



avg_range_make = (data.groupby('Make')['Electric_Range'].mean().sort_values(as

fig = px.bar(avg_range_make,x='Make',y='Electric_Range',color='Electric_Range'
             title='<b>Average Range by Manufacturer</b>')

fig.update_traces(marker_line_color='black', marker_line_width=1.5)

#Convert to HTML and display INSIDE notebook
HTML(fig.to_html())
```

Out[31]:

Top 10 Vehicle Makes by Average Electric Range

A bar chart was created to display the top 10 vehicle manufacturers with the highest average electric range by filtering out zero values, grouping by make, and calculating the mean range for each.

**Observations*

-Tesla has the highest average electric range (~240 miles)

-Jaguar and Polestar closely follow with similarly high ranges

-There is a significant drop after the top 3 brands

-Chevrolet holds a mid-range position (~150 miles)

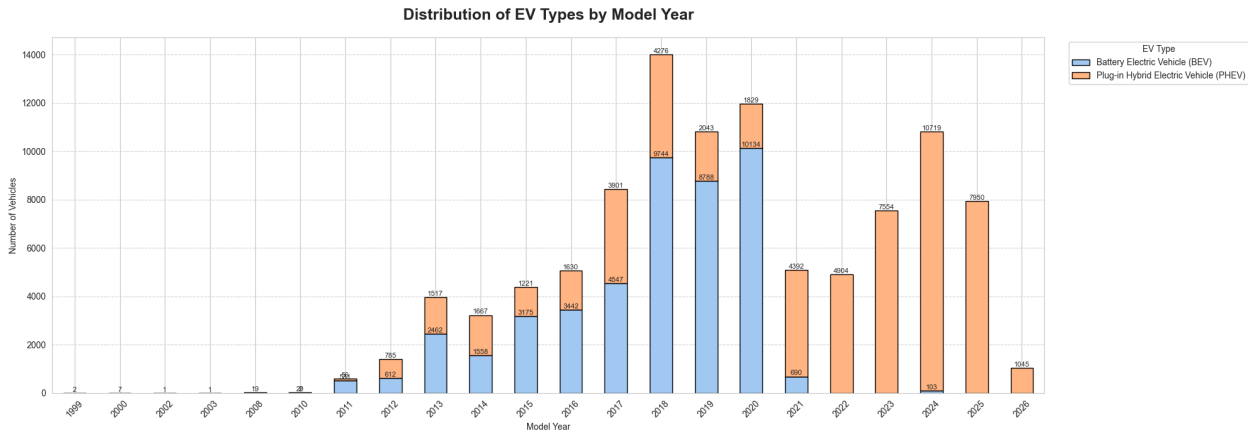
-Brands like Volkswagen, Nissan, and others have moderate ranges

-Fiat has the lowest average range among the top 10

Overall, premium EV brands tend to offer higher electric ranges compared to others.

15.EV Count by Year & Type

In [65]:
```
import matplotlib.pyplot as plt

ev_type_year = data.groupby(['Model_Year','Electric_Vehicle_Type']).size().uns

colors = sns.color_palette("pastel", n_colors=len(ev_type_year.columns))

ax = ev_type_year.plot(kind= 'bar',stacked=True,color =colors,edgecolor='black

for container in ax.containers:
    for bar in container:
        height = bar.get_height()
        if height > 0:  # avoid zero labels
            ax.text(bar.get_x() + bar.get_width()/2,bar.get_y() + height + 2,i

ax.set_title('Distribution of EV Types by Model Year', fontsize=18,fontweight=
ax.set_xlabel('Model Year')
ax.set_ylabel('Number of Vehicles')

plt.xticks(rotation=45)
plt.legend(title='EV Type', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```



Distribution of EV Types by Model Year

A stacked bar chart was created to show the distribution of Battery Electric Vehicles (BEV) and Plug-in Hybrid Electric Vehicles (PHEV) across different model years by grouping and counting each type.

**Observations*

-EV adoption was very low before 2011 and gradually increased over time

-Significant growth is observed from 2018 onwards

-The highest number of EVs is seen around 2023

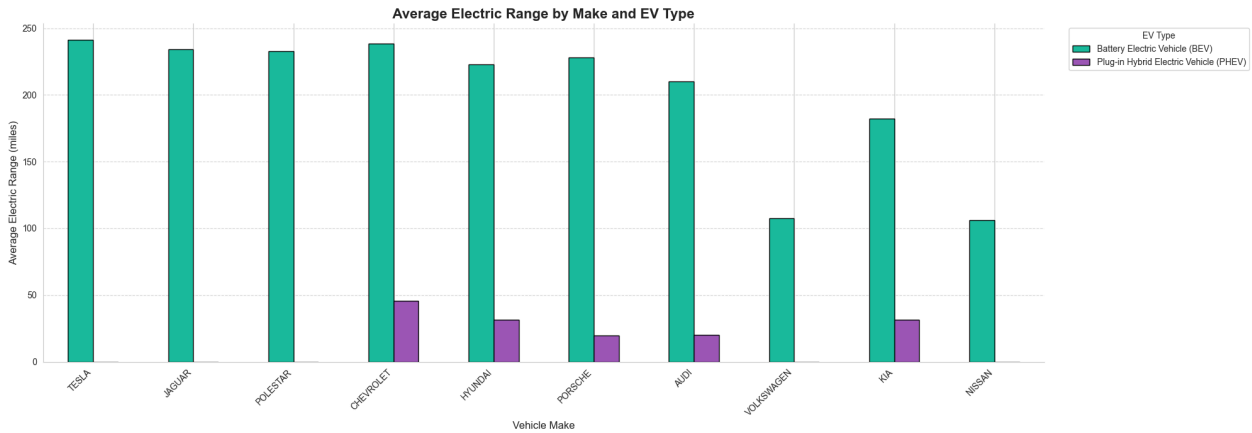
-Slight decline or variation is seen in the most recent years, possibly due to incomplete data

Overall, EV adoption has grown rapidly in recent years, with BEVs leading the trend.

16.Average Electric Range by Make and EV Type

In [32]:
```
import matplotlib.pyplot as plt

make_evtype_range = (data.groupby(['Make', 'Electric_Vehicle_Type'])['Electric

top_makes = make_evtype_range.mean(axis=1).sort_values(ascending=False).head(1
make_evtype_range = make_evtype_range.loc[top_makes.index]

colors =['#1ABC9C', '#9B59B6']
ax = make_evtype_range.plot(kind='bar',figsize=(20,7),color = colors,edgecolor

ax.set_title('Average Electric Range by Make and EV Type', fontsize=16, fontwe
ax.set_xlabel('Vehicle Make', fontsize=12)
ax.set_ylabel('Average Electric Range (miles)', fontsize=12)

plt.xticks(rotation=45, ha='right')
```

```
plt.legend(title='EV Type', bbox_to_anchor=(1.02, 1), loc='upper left')

plt.grid(axis='y', linestyle='--', alpha=0.7)

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()
```



Average Electric Range by Make and EV Type A bar chart was generated to compare the average electric range (in miles) across the top 10 vehicle makes, segmented by EV type (Battery Electric Vehicle vs. Plug-in Hybrid Electric Vehicle).

**Observations*

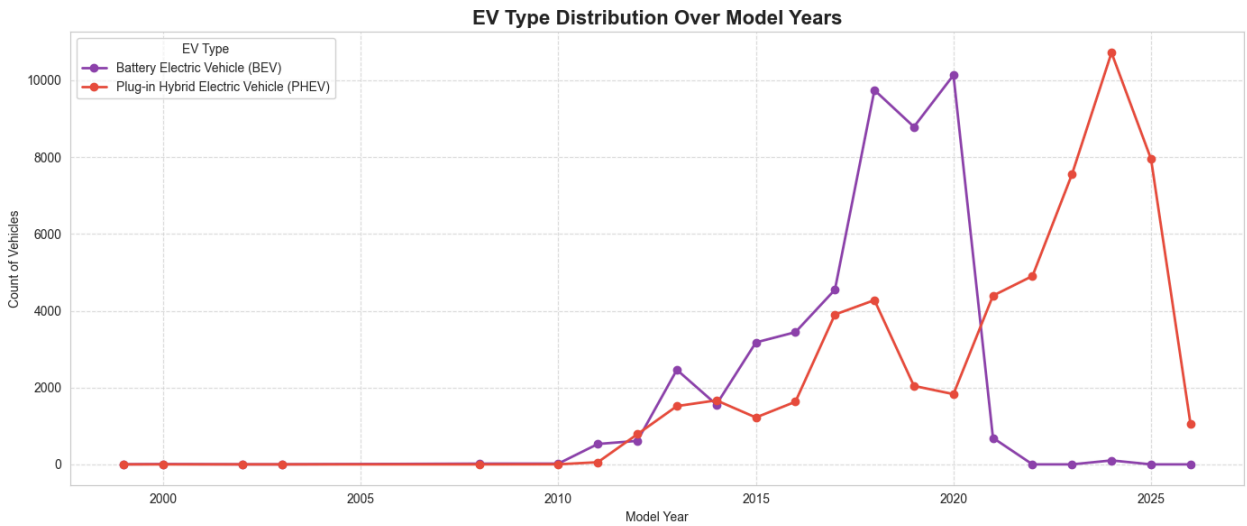
- TESLA, JAGUAR, and POLESTAR lead with the highest BEV ranges (~240–250 miles).
- Manufacturers offering both BEVs and PHEVs (CHEVROLET, HYUNDAI, PORSCHE, AUDI, KIA) consistently show higher ranges for BEVs compared to PHEVs.
- VOLKSWAGEN and NISSAN only display BEV data, with moderate ranges relative to leaders.
- PHEVs generally have shorter ranges, highlighting their reliance on hybrid systems rather than full electric capability.

In short, the chart underscores BEVs as range leaders, with TESLA and premium brands setting benchmarks, while PHEVs remain limited in electric-only distance

17.EV Type Distribution Over Model Years

In [33]:
```
import matplotlib.pyplot as plt
```

```
import pandas as pd

pivot2 = pd.pivot_table(data,index='Model_Year',columns='Electric_Vehicle_Type

colors = ['#8E44AD', '#E74C3C']
ax = pivot2.plot(figsize=(14,6),marker='o',linewidth=2,color=colors)

ax.set_title('EV Type Distribution Over Model Years', fontsize=16, fontweight=
ax.set_xlabel('Model Year')
ax.set_ylabel('Count of Vehicles')

plt.grid(True, linestyle='--', alpha=0.6)

plt.legend(title='EV Type')

plt.tight_layout()
plt.show()
```



EV Type Distribution Over Model Years

A line chart was plotted to show adoption trends of Battery Electric Vehicles (BEVs) and Plug-in Hybrid Electric Vehicles (PHEVs) from 1998–2026.

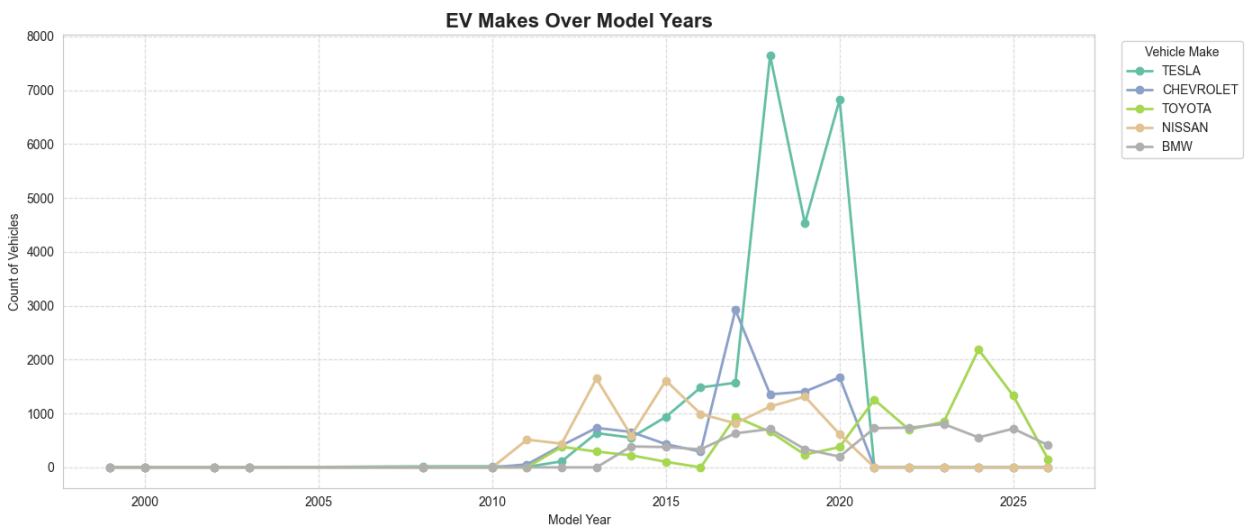
**Observations*

- BEVs surged after 2018, peaking in 2023 with over 50,000 vehicles, then declined.
- PHEVs peaked lower (~11,000 in 2024) before tapering off.
- BEVs dominate recent growth, while PHEVs remain a smaller segment.

In short, BEVs drive the majority of EV adoption, with PHEVs playing a secondary role.

18. EV Growth over Years

In [34]:
```
import matplotlib.pyplot as plt
import pandas as pd

pivot2 = pd.pivot_table(data,index='Model_Year',columns='Make',aggfunc='size')

top_5_makes = pivot2.sum().sort_values(ascending=False).head().index
pivot2 = pivot2[top_5_makes]

ax = pivot2.plot(figsize=(14,6),marker='o',linewidth=2,colormap='Set2')

ax.set_title('EV Makes Over Model Years', fontsize=16, fontweight='bold')
ax.set_xlabel('Model Year')
ax.set_ylabel('Count of Vehicles')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(title='Vehicle Make', bbox_to_anchor=(1.02, 1), loc='upper left')

plt.tight_layout()
plt.show()
```



EV Makes Over Model Years

A line chart was plotted to show the top five EV manufacturers (Tesla, Chevrolet, Nissan, Ford, Kia) across model years from before 2000 through 2026.

**Observations*

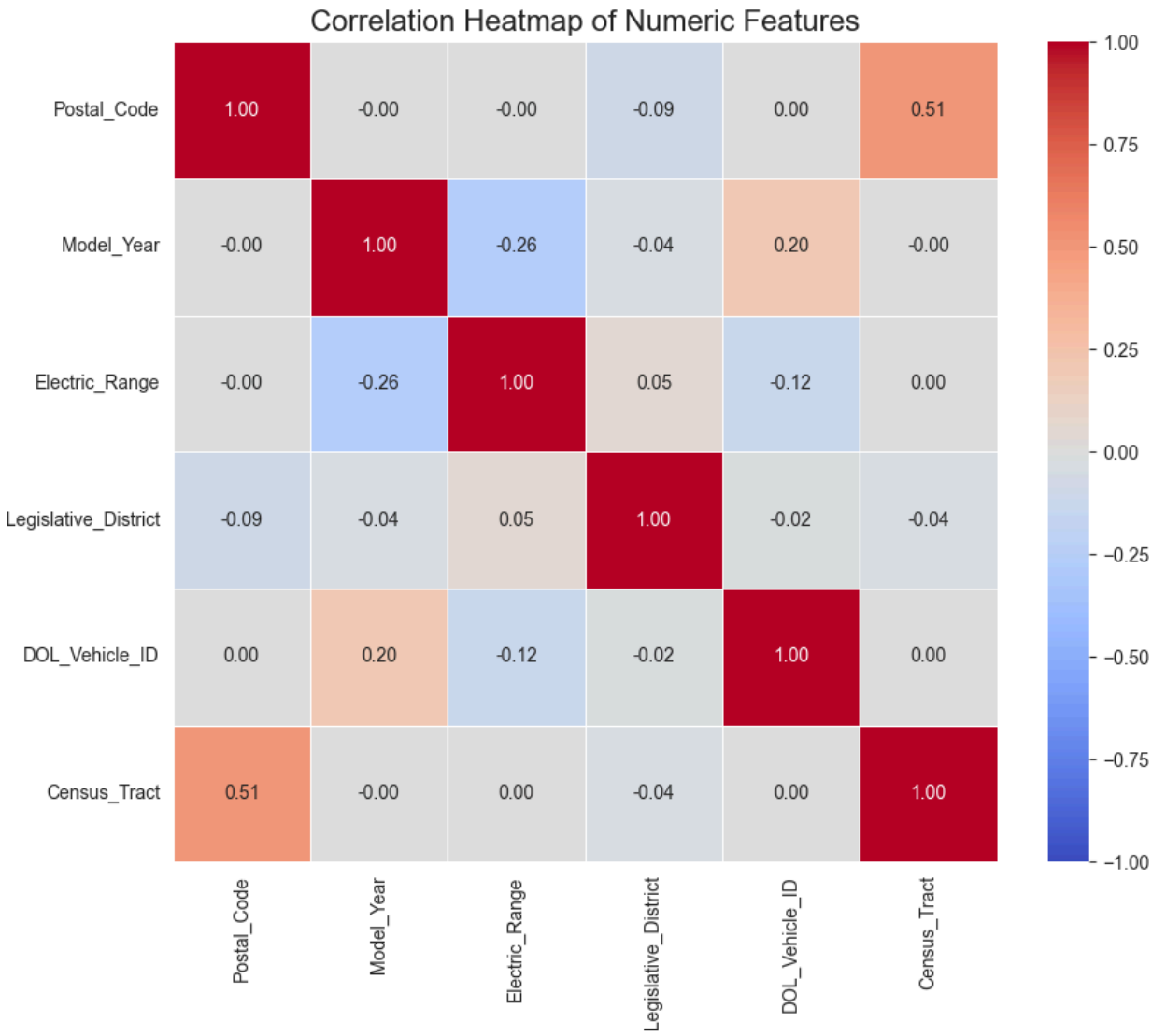
- Tesla dominates, surging after 2017 and peaking in 2023 with over 25,000 vehicles.
- Chevrolet, Nissan, Ford, and Kia remain comparatively lower, with modest peaks around 2022–2024.

- Tesla’s trajectory highlights its market leadership, while other makes show steady but limited growth.

In short, Tesla drives the bulk of EV adoption, with other manufacturers contributing smaller shares.

19. EV Dataset: Numeric Feature Correlations

In [34]:
```
import seaborn as sns
import matplotlib.pyplot as plt

numeric_cols = data.select_dtypes(include=['number'])

numeric_cols = data.select_dtypes(include=['number']).copy()
numeric_cols = numeric_cols[numeric_cols['Electric_Range'] != 0]


corr_matrix = numeric_cols.corr()


plt.figure(figsize=(10, 8))

sns.heatmap(corr_matrix, annot=True,fmt=".2f",cmap='coolwarm', cbar=True,linew

# Add title
plt.title('Correlation Heatmap of Numeric Features', fontsize=16)

# Show plot
plt.show()
```



Correlation Heatmap of Numeric Features

A correlation heatmap was generated to examine relationships among numeric variables such as Model Year, Electric Range, Legislative District, DOL Vehicle ID, Census Tract, and Vehicle Age.

**Observations*

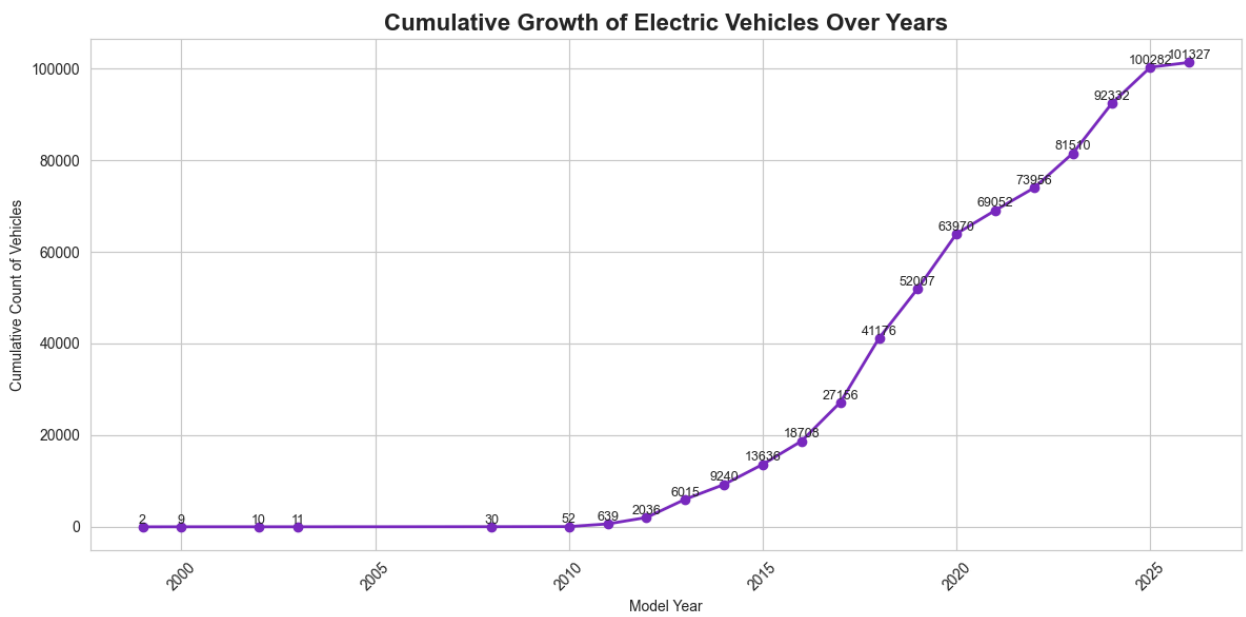
- Model Year and Vehicle Age show a perfect negative correlation (-1.00).
- Electric Range has weak correlations with most other features, indicating independence.
- Identifier-like variables (Legislative District, DOL Vehicle ID, Census Tract) show little correlation with others.
- Redundancy exists between Model Year and Vehicle Age, suggesting

one could be dropped to avoid multicollinearity.

In short, the heatmap highlights strong redundancy in time-related features and independence of range-related values.

20.Cumulative EV Growth

In [60]:
```
import matplotlib.pyplot as plt

cum_ev = data['Model_Year'].value_counts().sort_index().cumsum()

plt.figure(figsize=(12,6))

plt.plot(cum_ev.index,cum_ev.values,marker='o',linewidth=2,color = '#7B2CBF')


for x, y in zip(cum_ev.index, cum_ev.values):
    plt.text(x, y + 50,str(int(y)), ha='center',va='bottom',fontsize=9)


plt.title('Cumulative Growth of Electric Vehicles Over Years', fontsize=16, fc
plt.xlabel('Model Year')
plt.ylabel('Cumulative Count of Vehicles')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Cumulative Growth of Electric Vehicles Over Years

A line graph was generated to visualize the cumulative adoption of electric vehicles from 2000 to 2023. Each year's cumulative count was plotted and annotated to

highlight the rapid acceleration in EV uptake.

**Observations*

- Early adoption (2000–2007) was minimal, with counts below 100.
- A sharp surge began in 2008, marking the start of exponential growth.
- Major jumps occurred in 2010 (6,016) and 2020 (174,687), reflecting market expansion and policy support.
- Growth continued steadily, surpassing 270,000 cumulative vehicles by 2023.
- The trajectory suggests strong momentum, with adoption accelerating in recent years.

In short, the visualization highlights the exponential rise of EVs, with inflection points around 2008 and 2010, and sustained growth through the 2020s.

MAJOR INSIGHTS

Adoption & Growth Trends

Exponential Growth: After minimal adoption from 2000–2007, a sharp surge began in 2008, marking the start of exponential EV growth.

Key Inflection Points: Significant jumps in adoption occurred in 2010 (6,016 vehicles) and 2020 (174,687 vehicles), likely reflecting market expansion and increased policy support.

Recent Momentum: The population surpassed 270,000 cumulative vehicles by 2023, with sustained growth continuing through the 2020s.

Vehicle Types & Performance

BEV Dominance: Battery Electric Vehicles (BEVs) drive the majority of adoption, peaking at over 50,000 vehicles in 2023.

PHEV Role: Plug-in Hybrid Electric Vehicles (PHEVs) remain a secondary segment, peaking at approximately 11,000 vehicles in 2024.

Range Characteristics: Premium EV brands tend to offer higher electric ranges compared to other manufacturers.

Range Leaders: Tesla leads with the highest average electric range, while brands

like Fiat have the lowest average range among top manufacturers.

Market & Geographic Distribution

Top Manufacturers: Tesla, Chevrolet, and Nissan are the most common manufacturers in the dataset.

Popular Models: The Tesla Model Y and Model 3, along with the Nissan Leaf, are among the most frequently registered models.

Distribution Focus: The study highlights the variation of EV distribution across different counties and cities within Washington State to aid in infrastructure planning.

Project Significance

Environmental Impact: The analysis underscores the role of EVs in reducing carbon emissions and promoting sustainable transportation solutions.

Strategic Planning Understanding these adoption patterns is critical for planning future charging infrastructure and making informed policy decisions.

FUTURE DIRECTIONS AND OPPORTUNITIES

Infrastructure Expansion: Strategic planning and investment in charging infrastructure are crucial to support continued EV growth and longer-range vehicles.

Policy Implications: Supportive policies and incentives are vital to accelerate adoption, overcome barriers, and ensure equitable access to EVs and charging.

Research & Development: Continued innovation in battery technology, charging speeds, and sustainable manufacturing will drive the next wave of EV advancements.

Stakeholder Collaboration: Manufacturers, utility providers, urban planners, and governments must collaborate to build a cohesive and efficient EV ecosystem